

8교시: 구름 EXP 배움일기 - MSA 토폴로지와 연결 실패 증거

Week 3 Day 1 Lesson 8

보충 실습과 진도 회복

막힌 지점을 기준으로 다음 행동과 evidence를 남기고, 재실행 → 확인 → 진행합니다.

사용 방법

- 1) 막힌 지점 확인
- 2) 다음 행동 실행
- 3) 증거 수집 (evidence)
- 4) 재실행 후 확인
- 5) 진행 - 또는 질문 준비

막힌 지점	다음 행동	evidence (업데이트 증거)	재실행	질문 준비	cleanup
1 설치에서 막힘 (dependency / 패키지 설치 오류)	• 에러 로그의 첫 번째 실패 원인 확인 • Python 버전/가상환경/패키지 버전 확인 • requirements.txt 또는 명령 재실행	• 에러 실패 전체 로그 • Python -V, pip -V • 패키지 버전 목록 (pip list)	<pre>pip install -r requirements.txt</pre> 또는 설정한 설치 명령	• 어떤 명령에서 실패했는가? • 전체 에러 메시지 • 시도한 해결 방법	• 가상환경 정리 • pip 캐시 정리 • 재설치 준비
2 docker compose 설정에서 막힘 (compose config 오류)	• compose 파일 문법/틀어쓰기/버전 확인 • 환경변수(.env) 값 확인 • docker compose config 재실행	• docker compose config 출력 • .env 파일 내용 (민감정보 제외) • compose 파일 경로/버전	<pre>docker compose config</pre> 또는 <pre>docker compose up -d</pre>	• 어떤 파일/라인에서 오류? • 사용 중인 Compose 버전? • 변경한 부분은?	• docker compose down • 불필요한 네트워크/볼륨 정리 • .env 값 재확인
3 빌드에서 막힘 (이미지 빌드 오류)	• Dockerfile 오류 로그 확인 • 베이스 이미지/의존성 설치 확인 • 빌드 캐시 없이 재빌드	• docker build 전체 로그 • Dockerfile 경로/주요 라인 • base image 태그	<pre>docker build --no-cache -t <image>:dev .</pre>	• 어느 단계에서 실패? • 어떤 의존성/파일이 문제? • 변경한 내용은?	• 중간 이미지/빌드 캐시 정리 • docker system prune (선택) • 다시 빌드 준비
4 실행 중(runtime)에서 막힘 (컨테이너 기동/서비스 오류)	• 컨테이너 상태 및 로그 확인 • 포트 충돌/환경변수/볼륨 권한 확인 • 서비스 의존성(DB/Redis 등) 확인	• docker compose ps • docker compose logs --tail=200 • 에러 스크린샷/메시지	<pre>docker compose up -d</pre> 또는 실패한 서비스만 재시작	• 어떤 서비스가 다운? • 로그의 핵심 에러는? • 재현 절차는?	• docker compose down • 볼륨/네트워크 정리 • 설정/환경 정리
5 브라우저 확인에서 막힘 (접속/화면/기능 확인 실패)	• URL/포트/경로 확인 • 네트워크/프록시/CORS 확인 • 브라우저 개발자도구로 오류 확인	• 브라우저 오류 스크린샷 • Network/Console 로그 • 요청/응답 헤더 및 상태코드	브라우저 새로고침(강력 새로고침) 또는 다른 브라우저로 재확인	• 어떤 화면/동작에서 실패? • 에러 메시지/상태코드? • 재현 방법론?	• 캐시/쿠키 삭제 • 불필요한 탭/세션 정리 • 환경 초기화

오늘 회복 목표

- ☐ 1단계(설치) 통과
- ☐ 2단계(설정) 통과
- ☐ 3단계(빌드) 통과
- ☐ 4단계(실행) 통과
- ☐ 5단계(브라우저 확인) 통과

기록 (오늘의 회복 요약 / 어떤 조치가 효과적이었는가?)

다음 액션 (내일 할 일)

수업 목표

- Day1에서 확인한 MSA 토폴로지와 실행 증거를 정리한다.
- 구름 EXP 배움일기에 단순 감상이 아니라 다음 수업에서 다시 쓸 수 있는 evidence를 남긴다.
- Day2 장애 전파, health, timeout/retry 수업으로 넘어갈 질문을 만든다.

Day1 학습 서머리

오늘은 MSA를 “서비스를 작게 나누는 개발 방식”이 아니라 “여러 실행 단위를 운영하는 구조”로 봤다.

오늘 배운 내용	설명할 수 있어야 하는 문장
Monolith vs MSA	MSA는 독립 배포를 가능하게 하지만 network dependency와 운영 비용을 만든다.
Service contract	service별 image/build, port, env, dependency, health, logs를 표로 정리해야 한다.
frontend 경로	browser는 localhost:18083으로 들어오고 frontend가 api:8080으로 넘긴다.
API readiness	API process가 살아 있어도 DB 연결이 실패하면 /health는 실패할 수 있다.
worker 경로	worker는 host port가 없고 내부에서 api:8080/api/status를 호출한다.
DB dependency	API는 DB_HOST=db, DB_PORT=5432로 PostgreSQL에 붙는다.
장애 증거	API 중지, DB 중지, DB_HOST 오류는 서로 다른 로그와 status를 만든다.

오늘 남겨야 할 Evidence

다음 명령 중 최소 3개는 실제 출력 요약을 남긴다.

```
docker compose config
docker compose ps
curl -s http://localhost:18083/api/status
curl -s http://localhost:18084/health
docker compose logs --tail=60 api
docker compose logs --tail=60 worker
```

출력 전체를 붙일 필요는 없다. 핵심 줄만 정리한다.

명령	남길 핵심
docker compose ps	service별 Up/healthy 상태
frontend curl	frontend_to_api, database_reachable
API health	ready, db_host, error
API logs	request path, request id
worker logs	api_url, status 또는 error

구름 EXP 배움일기 작성 가이드

항목	작성 내용
오늘의 구조	frontend -> api -> db, worker -> api -> db 흐름
가장 헷갈린 지점	host port/container port/service name/env/health 중 하나
실행 증거	명령 1개와 핵심 출력 요약
장애 증거	API 중지 또는 DB 중지 시 본 증상
내 판단	어떤 service를 먼저 의심했는지와 이유
Day2 질문	health, readiness, timeout/retry에서 더 확인하고 싶은 것

작성 예시

오늘은 msa-demo를 실행해서 frontend, api, worker, db의 역할을 확인했다.
 사용자는 localhost:18083으로 들어오지만 frontend container는 api:8080으로 요청을 넘긴다.
 api는 DB_HOST=db로 PostgreSQL에 연결하고, /health에서 database_reachable 상태를 보여준다.
 API를 중지했을 때 frontend 경로와 worker 로그가 모두 실패해서 API 장애가 다른 service로 전파된다는 것을 확인했다.
 아직 헷갈리는 것은 container 내부에서 localhost를 쓰면 왜 자기 자신을 가리키는지다.
 Day2에서는 DB가 느리게 준비될 때 depends_on과 healthcheck만으로 충분한지 확인하고 싶다.

Day2로 넘길 질문

질문	Day2 연결
API가 running인데 /health가 503이면 장애인가	readiness
worker가 실패해도 사용자는 정상일 수 있는가	부분 장애
retry를 많이 하면 항상 좋은가	timeout/retry
여러 service 로그를 어떻게 이어 볼 것인가	correlation id
Compose의 depends_on은 Kubernetes에서 무엇으로 바뀌는가	readiness probe

Cleanup

Day2에서 같은 앱을 바로 쓸 수 있으므로 수업 환경에 따라 유지하거나 정리한다.

정리:

```
cd week3/day1/labs/msa-demo  
docker compose down
```

DB volume까지 초기화:

```
docker compose down -v
```

핵심 포인트

Day1의 완료 기준은 MSA라는 말을 외우는 것이 아니다. msa-demo의 service contract를 보고 사용자 요청과 내부 dependency를 설명하며, 실패했을 때 어느 service의 어떤 증거를 먼저 볼지 말할 수 있어야 한다.